TUTORIAL – 12

AI-Powered Software Testing

Aim

To analyze AI-powered software testing tools and apply Generative AI techniques for requirement analysis, test-case generation, test-data creation, defect prediction, and Selenium-based automated testing of a login module.

Software / Tools Required

PyCharm IDE

Python 3.x

Google Chrome browser

Selenium WebDriver package

Generative AI assistant

Internet connection

Requirement / Problem Statement

A registered user should be able to log in using a valid username and password. Invalid credentials should be rejected and an appropriate error message should be displayed.

Test Cases and Test Data

Test Case ID	Scenario	Test Data	Expected Result
TC-12-01	Valid login	standard_user / secret_sauce	Products page opens

TC-12-02	Invalid password	standard_user / wrong123	Error message is displayed
TC-12-03	Invalid username	wrong_user / secret_sauce	Error message is displayed
TC-12-04	Empty username	Blank / secret_sauce	Validation message is displayed
TC-12-05	Empty password	standard_user / Blank	Validation message is displayed

Script / Source Code

Create a Python file named ai_login_test.py in PyCharm and paste the reviewed code below.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions
as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get("https://www.saucedemo.com/")
    wait = WebDriverWait(driver, 10)

    username = wait.until(
        EC.visibility_of_element_located((By.ID, "user-name"))
    )
    password = driver.find_element(By.ID, "password")
```

```
    login_button = driver.find_element(By.ID, "login-button")

    username.send_keys("standard_user")
    password.send_keys("secret_sauce")
    login_button.click()

    wait.until(EC.url_contains("inventory"))
    assert "inventory" in driver.current_url

    print("AI-Assisted Selenium Test: PASS")
    print("Requirement Validation: PASS")

except Exception as error:
    print("AI-Assisted Selenium Test: FAIL")
    print("Reason:", error)

finally:
    driver.quit()
```

Procedure (PyCharm)

1. Install Python 3.x if it is not already installed. During installation, enable “Add Python to PATH” if using the Windows installer.
2. Install and open PyCharm. Create a new Python project named AI_Software_Testing and select a valid Python 3.x interpreter.
3. Open the project and create a Python file named ai_login_test.py.
4. Open PyCharm’s Terminal tool window and install Selenium using: python -m pip install selenium. If needed on Windows, use: py -m pip install selenium.

Paste the source code from this report into ai_login_test.py and save the file.

Ensure Google Chrome is installed and internet access is available. Selenium Manager normally obtains the compatible Chrome driver automatically.

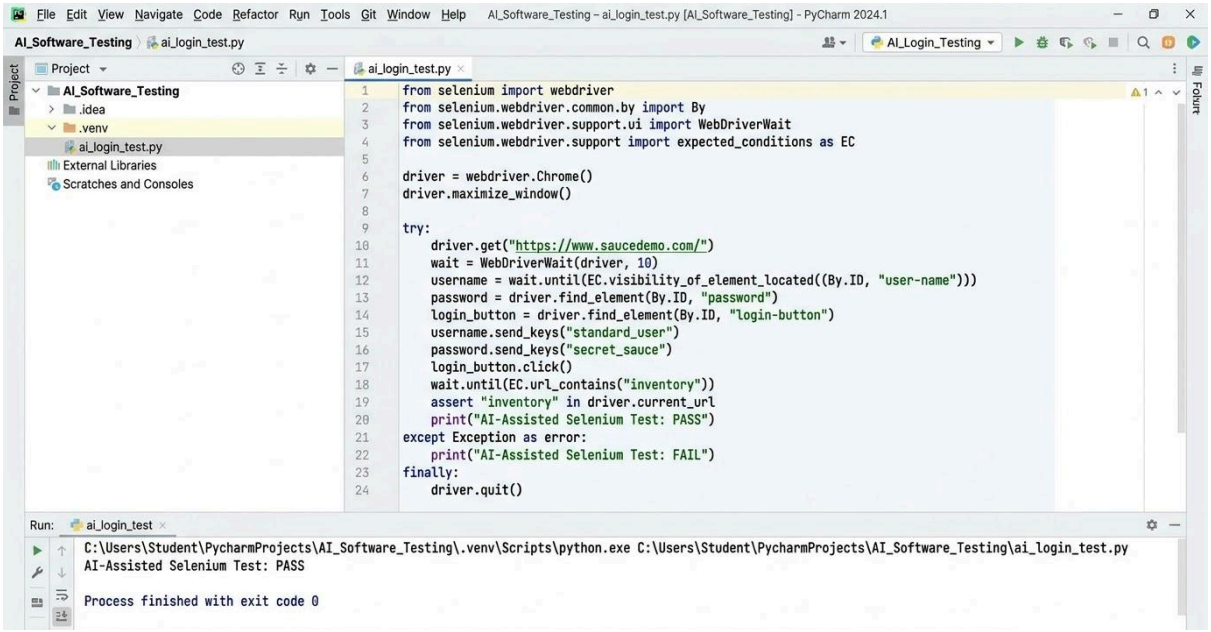
Right-click ai_login_test.py and select Run 'ai_login_test', or use the green Run button.

Observe Chrome opening SauceDemo, submitting the demo credentials, and navigating to the Products page. Check the Run/Console window for PASS messages.

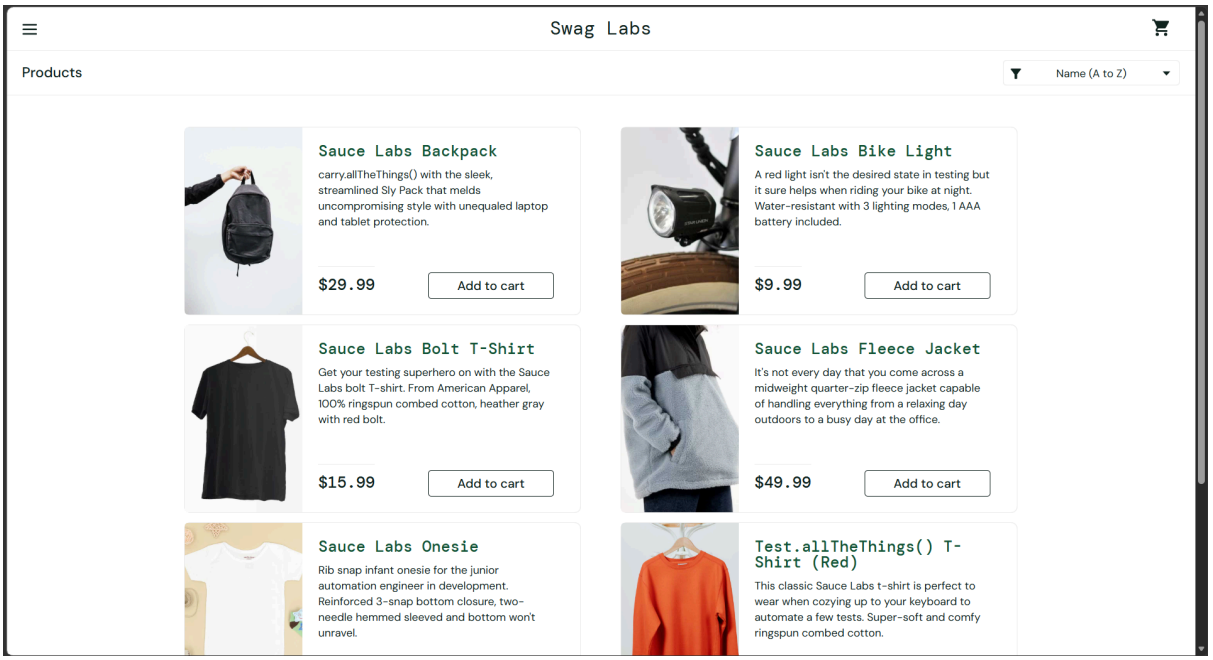
If the script fails, review the error message, check Python/Selenium/Chrome setup, correct the issue, and run the file again.

Output Screenshot

Screenshot 1 – Source Code in PyCharm



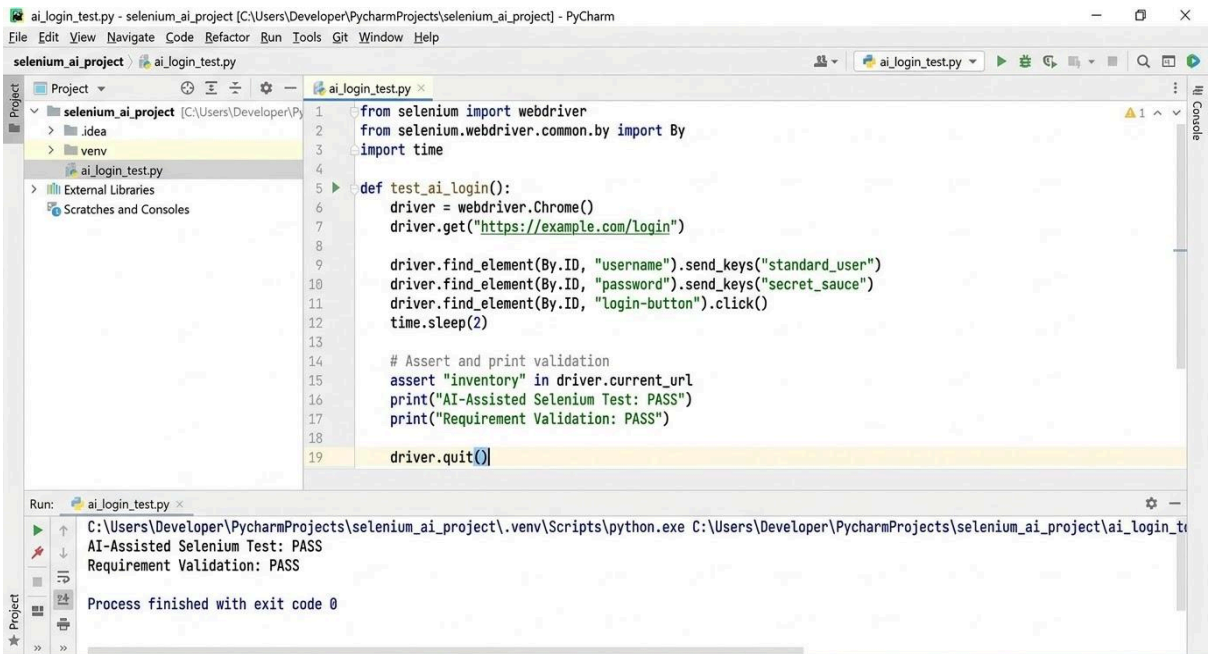
Screenshot 2 – SauceDemo Browser Output



.

Screenshot 3 – PyCharm Run / Console Output

.



Conclusion

This practical demonstrates how Generative AI can assist with requirement analysis, test-case and test-data preparation, defect prediction, and Selenium automation. AI-generated output should be reviewed and executed to verify its behavior. The provided script automates the valid-login scenario; negative and empty-field scenarios should also be tested separately.